

APPLICATION SECURITY ASSESSMENT

Web Application Security Assessment Report

OWASP Juice Shop
Local Docker Deployment

TARGET APPLICATION	OWASP Juice Shop
TARGET URL	http://127.0.0.1:3000
ENVIRONMENT	Local Docker container
ASSESSMENT TYPE	Web application security assessment
ASSESSMENT DATE	24 September 2026
RESEARCHER	Aryan Singh
CLASSIFICATION	Confidential – Educational / Portfolio Use

This report documents confirmed laboratory findings, source-level observations, and tests that did not demonstrate exploitation. Live authentication tokens and secrets are omitted.

Document Control

Document title	Web Application Security Assessment Report
Target	OWASP Juice Shop
Environment	Local Docker deployment
Target address	http://127.0.0.1:3000
Assessment date	24 September 2026
Researcher	Aryan Singh
Assessment type	Black / grey-box web assessment with source review
Classification	Confidential – Educational / Portfolio Use
Report status	Final laboratory technical report
Evidence package	Authorized local case-study workspace

Classification statement

This document is intended for educational, research, and portfolio use. It describes testing against an intentionally vulnerable local application. It is not a warranty of security for any other deployment.

Version

Version	Date	Notes
1.0	24 September 2026	Final technical assessment based on captured laboratory evidence.

1 Confidentiality and Authorization

This assessment was conducted against a locally hosted, intentionally vulnerable instance of OWASP Juice Shop. Testing was restricted to that local deployment. No external production system, third-party tenant, or remote infrastructure was targeted.

All exploitation evidence in this report was generated in the authorized laboratory environment. Live JSON Web Tokens, passwords, private keys, and other secrets are excluded or redacted.

The application is designed as a security training target. Findings therefore describe the tested laboratory build on 24 September 2026 and should not be treated as a complete security guarantee for other versions or production installations.

2 Executive Summary

The assessment combined service reconnaissance, HTTP baseline review, manual API testing, and static review of application source. Six issues were dynamically confirmed. Additional source-level conditions were recorded separately because static review alone does not prove exploitability in the running instance.

The most severe confirmed issue is SQL injection in the login endpoint. A crafted email value produced HTTP 200 and an administrative authentication context for `admin@juice-sh.op`. A separate unauthenticated registration request that included `role=admin` created user ID 26 with an administrative role. An authenticated customer (user ID 25) retrieved basket 1 belonging to user ID 1. The `/ftp/` path exposed a directory listing and served `acquisitions.md` without authentication. Invalid redirect input returned stack traces and internal build paths. Product search accepted input that reached a raw SQL statement and produced SQLite errors; successful data extraction was not demonstrated.

2.1 Confirmed findings

ID	Finding	Severity	Status
F-01	SQL injection authentication bypass	CRITICAL	CONFIRMED
F-02	Mass assignment privilege escalation	HIGH	CONFIRMED
F-03	Broken object-level authorization on baskets	HIGH	CONFIRMED
F-04	FTP directory listing and file exposure	MEDIUM	CONFIRMED
F-05	Verbose error and stack-trace disclosure	LOW MEDIUM	/ CONFIRMED
F-06	Product-search SQL injection behavior	HIGH	CONFIRMED

2.2 Security posture in the tested build

Authentication, registration privilege handling, and object-level authorization failed under laboratory testing. File exposure and verbose errors further reduced the effort required to map the application. Product-search SQL construction is unsafe, but this report does not claim database extraction from that endpoint.

3 Objectives

The assessment objectives were to:

- Identify reachable services and HTTP behavior on the local target.

- Test authentication, registration, and object-access controls.
- Validate input-handling behavior on search and related APIs.
- Assess public file exposure under `/ftp/`.
- Correlate dynamic results with source-code review where available.
- Separate confirmed exploitation from source-only hypotheses.

4 Scope and Environment

4.1 In-scope target

Application	OWASP Juice Shop
URL	http://127.0.0.1:3000
Host	127.0.0.1
Environment	Docker container
Image observed	bkimminich/juice-shop
Container status	running
Published ports	3000/tcp bound to 0.0.0.0 and ::
Testing boundary	Local Juice Shop deployment only
Authorization	Intentionally vulnerable laboratory application

4.2 Testing areas included

Reconnaissance, HTTP and API review, authentication, authorization, broken access control, SQL injection, file exposure, directory listing, error handling, security headers, source-code review, and container/deployment observations limited to the local instance.

4.3 Out of scope

- External production systems and third-party infrastructure
- Denial-of-service testing
- Destructive exploitation
- Credential attacks against external accounts
- Cracking or decrypting exposed laboratory artifacts

5 Methodology

The work followed a four-phase laboratory workflow.

5.1 Phase 1 – Reconnaissance

Nmap service detection was used against port 3000 on localhost. The host was up and 3000/tcp was open. Nmap labeled the service `ppp?`; that label was treated as a fingerprint artifact, not as a vulnerability. Docker inspection confirmed a running Juice Shop image publishing port 3000.

5.2 Phase 2 – HTTP baseline

An HTTP request to the application root returned HTTP 200. Response headers were recorded, including `Access-Control-Allow-Origin: *`, `X-Content-Type-Options: nosniff`, `X-Frame-Options:`

SAMEORIGIN, and Feature-Policy: `payment 'self'`. The wildcard CORS origin was not independently scored as a vulnerability because credentialed cross-origin abuse was not established from that header observation alone.

5.3 Phase 3 – Static analysis

Semgrep output and manual source review were used to locate query construction, object lookup logic, directory serving, error handling, registration resource configuration, and additional sensitive code paths. Source findings were not automatically promoted to confirmed vulnerabilities.

5.4 Phase 4 – Dynamic validation

Candidate issues were tested against the running application. Successful exploitation was captured in evidence files and correlated with source behavior. Failed or incomplete tests were recorded as not confirmed.

6 Tools

Tool	Use in this assessment
Nmap 7.991	Service detection on 127.0.0.1:3000
HTTP client (curl / equivalent)	Baseline requests, API tests, file retrieval
Docker inspection	Confirm local container image, status, and port bindings
Semgrep	JavaScript/TypeScript static analysis output
Manual source review	Correlation of dynamic evidence to application routes

7 Attack Surface

The tested instance exposed an HTTP application on port 3000. The following components were relevant to confirmed or attempted tests:

- Application root and static frontend
- POST `/rest/user/login`
- POST `/api/Users/`
- GET `/rest/basket/{id}`
- GET `/rest/products/search`
- GET `/ftp/` and files under that path
- GET `/redirect`

Source review additionally identified B2B order processing, XML and ZIP upload handling, profile-image URL fetching, and JWT key handling. Those items remain source-level unless independently validated.

8 Executive Findings Table

ID	Title	Severity	Endpoint
F-01	SQL injection authentication bypass	Critical	POST /rest/user/login
F-02	Mass assignment privilege escalation	High	POST /api/Users/
F-03	Basket BOLA / IDOR	High	GET /rest/basket/{id}
F-04	FTP listing and sensitive file exposure	Medium	GET /ftp/
F-05	Verbose error disclosure	Low / Medium	GET /redirect
F-06	Product-search SQL injection behavior	High	GET /rest/products/search

9 Detailed Findings

F-01 – SQL Injection Authentication Bypass

Severity: **CRITICAL** | Status: **CONFIRMED**

Affected component

POST /rest/user/login / routes/login.ts

Mapping

OWASP A03:2021 Injection; CWE-89 (SQL Injection); CWE-287 (Improper Authentication). These mappings describe the confirmed class of issue. No CVE identifier is assigned.

Description

The login handler builds a SQL query by interpolating the client-supplied email value into the statement. Dynamic testing showed that a crafted email altered query logic and produced a successful administrative authentication response.

Technical details / root cause

Source review shows the query constructed as:

```
1 SELECT * FROM Users
2 WHERE email = '<user input>'
3 AND password = '<password hash>'
4 AND deletedAt IS NULL
```

`req.body.email` is inserted directly rather than bound as a parameter. The Sequelize call uses a raw query string with `plain: true`, which returns the first matching user row.

Validation / evidence

Laboratory payload:

```
1 {"email":"' OR 1=1--","password":"anything"}
```

Observed result:

```
1 HTTP/1.1 200 OK
2 Email: admin@juice-sh.op
3 Role: admin
```

An authentication token was issued. The token value is omitted from this report. Evidence file: `login-sqli-auth-bypass.txt`.

Impact

In the tested laboratory build, an unauthenticated requester obtained an administrative session. Potential laboratory impact includes authentication bypass, access to administrative application functions, and compromise of application-level trust boundaries. No production business-loss figure is claimed.

Remediation

Use parameterized queries or ORM bindings. Do not concatenate user-controlled values into SQL strings. Validate authentication inputs and add regression tests for injection payloads.

Retest guidance

Repeat `POST /rest/user/login` with the recorded payload and with additional comment/boolean variants. Expected secure result: authentication failure and no token. Confirm the source no longer interpolates `req.body.email`.

F-02 – Mass Assignment Privilege Escalation

Severity: **HIGH** | Status: **CONFIRMED**

Affected component

`POST /api/Users/` / Finale REST resource for the User model

Mapping

OWASP A01:2021 Broken Access Control; CWE-915 (Improperly Controlled Modification of Dynamically-Determined Object Attributes).

Description

User registration accepted a client-supplied `role` property and created an account with administrative privileges. The request did not require authentication.

Technical details / root cause

Source review indicates the generated User API excludes `password` and `totpSecret` from serialized output but passes request body fields into model creation. The `role` attribute is therefore client-controllable. Server-side logic did not force `role=customer` during registration.

Validation / evidence

Request included `role` set to `admin` together with registration fields. Observed result:

```
1 HTTP/1.1 201 Created
2 User ID: 26
3 Email: massassignment-test-2026@example.com
4 Role: admin
```

Evidence files: `mass-assignment-test.json`, `mass-assignment-admin.txt`. The laboratory password used in testing is omitted from this report.

Impact

An unauthenticated requester created an administrative account. This bypasses intended registration privilege boundaries in the tested build.

Remediation

Allowlist registration fields. Assign role, account status, and authorization flags only in trusted server-side logic.

Retest guidance

Resubmit registration with `role=admin`. Expected secure result: account created as customer, or the extra field rejected.

F-03 – Broken Object-Level Authorization / IDOR

Severity: **HIGH** | Status: **CONFIRMED**

Affected component

GET `/rest/basket/{id}` / `routes/basket.ts`

Mapping

OWASP A01:2021 Broken Access Control; CWE-639 (Authorization Bypass Through User-Controlled Key); BOLA.

Description

An authenticated customer retrieved another user's basket by changing the basket identifier.

Technical details / root cause

`retrieveBasket()` reads `req.params.id` and queries `BasketModel.findOne` with only that identifier. Ownership is not enforced before the object is returned. Challenge-tracking logic detects cross-user access but does not deny the response.

Validation / evidence

Authenticated test user:

```
1 User ID: 25
2 Role: customer
3 Own basket: 6
```

Own object:

```
1 GET /rest/basket/6
2 HTTP/1.1 200 OK
3 UserId: 25
```

Other user's object:

```
1 GET /rest/basket/1
2 HTTP/1.1 200 OK
3 UserId: 1
```

Evidence files: `idor-basket.txt`, `idor-root-cause.txt`. Bearer tokens used during the test are omitted.

Impact

A low-privileged authenticated user can read another user's basket contents in the tested build. This report does not claim that every object type in the application is affected.

Remediation

Verify ownership or an explicit authorization grant before returning the object. Return HTTP 403 when the authenticated user does not own the requested basket and is not authorized.

Retest guidance

Authenticate as a customer and request another user's basket ID. Expected secure result: HTTP 403 or equivalent denial, with no foreign basket body.

F-04 – FTP Directory Listing and Sensitive File Exposure

Severity: **MEDIUM** | Status: **CONFIRMED**

Affected component

GET /ftp/, GET /ftp/acquisitions.md

Mapping

OWASP A01:2021 Broken Access Control; CWE-548 (Exposure of Information Through Directory Listing); CWE-200 (Exposure of Sensitive Information).

Description

Unauthenticated requests received a directory listing under `/ftp/` and retrieved a document marked confidential in the laboratory application.

Technical details / root cause

Source review shows `serveIndex` mounted on `/ftp` and static file serving for allowlisted extensions such as `.md` and `.pdf`. Sensitive laboratory files are stored in that public directory.

Validation / evidence

```
1 GET /ftp/  
2 HTTP/1.1 200 OK  
3 listing directory /ftp/
```

Observed names included: `incident-support.kdbx`, `package.json.bak`, `package-lock.json.bak`, `encrypt.pyc`, `announcement_encrypted.md`, `suspicious_errors.yml`, `acquisitions.md`, `legal.md`.

```
1 GET /ftp/acquisitions.md  
2 HTTP/1.1 200 OK
```

The retrieved Markdown identified itself as confidential and contained planned-acquisition content in the laboratory application. `announcement_encrypted.md` was also reachable (HTTP 200, `Content-Type: text/markdown`, length 369237). That artifact was not decrypted, per scope restrictions.

Impact

Unauthenticated enumeration of filenames and retrieval of at least one confidential laboratory document. Backup and credential-store filenames were visible; those artifacts were not attacked further.

Remediation

Disable directory indexing. Remove backup and sensitive files from web-accessible paths. Require authorization for remaining file access.

Retest guidance

Request `/ftp/` and `/ftp/acquisitions.md` without authentication. Expected secure result: listing disabled and unauthorized file access denied.

F-05 – Verbose Error and Stack Trace Disclosure

Severity: **LOW / MEDIUM** | Status: **CONFIRMED**

Affected component

GET `/redirect?to=invalid` / error-handling middleware

Mapping

OWASP A04:2021 Insecure Design / security misconfiguration class; CWE-209 (Generation of Error Message Containing Sensitive Information).

Description

An invalid redirect target returned HTTP 406 with internal paths, build filenames, and stack frames.

Technical details / root cause

Source review shows `errorhandler()` registered in the Express pipeline. Disallowed redirect targets are passed to the error handler, which rendered detailed error output to the client.

Validation / evidence

```
1 GET /redirect?to=invalid
2 HTTP/1.1 406 Not Acceptable
```

Exposed path examples:

```
1 /juice-shop/build/routes/redirect.js
2 /juice-shop/build/routes/verify.js
3 /juice-shop/build/lib/insecurity.js
```

Evidence file: `verbose-error-disclosure.txt`.

Impact

The response discloses implementation structure that can assist further testing. It is not, by itself, a demonstrated authentication or data-store compromise.

Remediation

Return generic client errors in non-development mode. Log stack traces server-side only.

Retest guidance

Repeat the invalid redirect request. Expected secure result: generic error body without filesystem paths or stack frames.

F-06 – Product Search SQL Injection Behavior

Severity: **HIGH** | Status: **CONFIRMED**

Affected component

GET `/rest/products/search` / `routes/search.ts`

Mapping

OWASP A03:2021 Injection; CWE-89 (SQL Injection). Mapping applies to unsafe query construction. This report does not claim completed extraction.

Description

Search criteria are inserted into a SQL statement. Dynamic tests produced SQLite errors. Boolean-based data extraction was not demonstrated with the payloads used.

Technical details / root cause

Source review shows `req.query.q` interpolated into:

```
1 SELECT * FROM Products
2 WHERE ((name LIKE '%<criteria>%'
3      OR description LIKE '%<criteria>%'))
4      AND deletedAt IS NULL)
5 ORDER BY name
```

Validation / evidence

Baseline:

```
1 GET /rest/products/search?q=zzzzzz
2 HTTP 200, empty data array
```

Injection-style requests:

```
1 GET /rest/products/search?q=' OR '1'='1' --
2 HTTP 500
3 SQLITE_ERROR: incomplete input
4
5 GET /rest/products/search?q=' AND '1'='2' --
6 HTTP 500
7 SQLITE_ERROR: incomplete input
```

Evidence file: `sql-injection-search.txt`.

Assessment limitation

These results confirm unsafe SQL construction and error-producing behavior. They do not establish successful database extraction. This report therefore does not claim extraction of users, hashes, or schema data through this endpoint.

Impact

User input can alter SQL parser behavior and disclose database error text. Residual risk of further injection exists because the query is concatenated, but that further exploitation was not completed here.

Remediation

Replace concatenation with parameterized queries or ORM filters. Do not return raw database errors to clients.

Retest guidance

Replay the recorded payloads and any later balanced-parenthesis tests only in the isolated laboratory. Expected secure result: no SQL error leakage and equivalent results for equivalent search terms without query alteration.

10 Tested but Not Confirmed

The following items were tested or observed during the laboratory work. They are not reported as confirmed vulnerabilities.

10.1 External open redirect – not confirmed

An external redirect target was rejected during testing. The invalid local test (`to=invalid`) produced a 406 error rather than an open redirect to an external site. Open redirect exploitation is therefore not claimed.

10.2 Direct access to backup filenames – not confirmed as download

Directory listing exposed `package.json.bak` and `package-lock.json.bak`. Direct retrieval of those `.bak` objects was rejected in the tested case. Filename exposure remains part of F-04; successful backup-file download is not confirmed.

10.3 Unauthenticated basket access – not confirmed

Basket retrieval without authentication returned HTTP 401 in the tested case. The confirmed issue is authorized-but-unauthorized-object access (F-03), not anonymous basket read.

10.4 Profile object IDOR – not confirmed

No evidence in the supplied package demonstrates successful unauthorized access to another user's profile object. Profile IDOR is therefore not confirmed.

10.5 Wildcard CORS header – not independently confirmed as exploitable

`Access-Control-Allow-Origin: *` was present on HTTP 200 responses. Credentialed cross-origin data access was not established from that observation alone and is not scored as a confirmed finding.

11 Source-Level Observations

The following items come from source review. They are **not** dynamically confirmed in this assessment and must not be treated as proven vulnerabilities until isolated laboratory validation is performed.

11.1 B2B order processing and VM evaluation

`routes/b2bOrder.ts` evaluates order-line data in a `vm` context. This is a source-level code-execution concern. It was not dynamically validated here.

11.2 XML parsing flags on file upload

`routes/fileUpload.ts` and `lib/xml.ts` enable XML parser options associated with external entity handling. XXE or related file disclosure was not dynamically demonstrated in this evidence pack.

11.3 Archive extraction / Zip Slip concern

ZIP handling concatenates archive entry paths into a destination directory with a project-root containment check. Zip Slip overwrite was not dynamically demonstrated here.

11.4 Profile image URL fetch

`routes/profileImageUrlUpload.ts` fetches a client-supplied image URL. SSRF against internal or metadata services was not dynamically demonstrated here.

11.5 JWT implementation details

Source review identified hardcoded key material and JWT verification behavior in `lib/insecurity.ts`. This report does not reproduce key material and does not claim completed token forgery testing beyond the separately confirmed login bypass.

11.6 Additional mass-assignment surfaces

Source review notes broader Finale/Epilogue model exposure. Only registration `role` assignment was dynamically confirmed (F-02).

If testing is extended, these items should be validated only against the isolated local instance.

12 Remediation Roadmap

12.1 Priority 1 – Authentication and privilege controls

1. Replace login SQL concatenation with parameterized queries.
2. Prevent client-controlled role assignment at registration.
3. Add regression tests for the recorded login and registration payloads.

12.2 Priority 2 – Object-level authorization

1. Enforce basket ownership before returning objects.
2. Return HTTP 403 for unauthorized object access.
3. Review other identifier-based endpoints using the same pattern.

12.3 Priority 3 – File exposure

1. Disable directory indexing on `/ftp`.
2. Remove backup and sensitive files from public paths.
3. Require authorization for remaining file access.

12.4 Priority 4 – Error handling and search SQL

1. Disable verbose error bodies outside development mode.
2. Parameterize product search and hide database errors.

12.5 Priority 5 – Secure development

1. Keep SAST in CI, but require dynamic confirmation before scoring.
2. Review VM evaluation, XML flags, archive extraction, and URL fetch before any non-lab deployment of similar code.

13 Retesting Strategy

Retest after each Priority 1 and Priority 2 change.

- Replay F-01 through F-06 payloads from the evidence files.
- Verify negative tests still fail closed: unauthenticated basket access, rejected external redirect, rejected `.bak` download.
- Do not treat a source-level observation as closed until a focused dynamic test is recorded.
- Keep tokens and passwords out of the retest report body.

14 Testing Matrix

Test area	Result	Observation
Reconnaissance	Completed	Port 3000 open on localhost.
HTTP baseline	Completed	HTTP 200 from application root.
Security headers	Reviewed	Common headers present; wildcard CORS not scored alone.
Login authentication	Failed	SQL injection bypass to admin (F-01).
User registration	Failed	Client <code>role=admin</code> created admin user 26 (F-02).
Basket authorization	Failed	Customer 25 read basket 1 (F-03).
Unauthenticated basket	Passed (tested case)	HTTP 401 without authentication.
FTP directory listing	Failed	Listing enabled (F-04).
Sensitive file access	Failed	<code>acquisitions.md</code> retrieved (F-04).
Direct <code>.bak</code> download	Passed (tested case)	Direct backup retrieval rejected.
Verbose errors	Failed	Stack traces and paths disclosed (F-05).
Product search SQL	Failed / limited	SQLite errors; extraction not shown (F-06).
External open redirect	Not confirmed	External target rejected.
Profile IDOR	Not confirmed	No successful profile-object abuse recorded.
Source-only paths	Not dynamically confirmed	VM, XML, ZIP, SSRF, JWT details.

15 Evidence Inventory

Supporting files from the laboratory workspace:

- `scans/nmap-3000.txt`
- `evidence/baseline-http.txt`
- `evidence/http-headers.txt`
- `evidence/baseline-docker.txt`
- `scans/semgrep-javascript.json`
- `evidence/login-sqli-auth-bypass.txt`
- `evidence/mass-assignment-admin.txt`
- `evidence/idor-basket.txt`
- `evidence/idor-root-cause.txt`
- `evidence/directory-listing-ftp.txt`
- `evidence/sensitive-file-exposure.txt`
- `evidence/verbose-error-disclosure.txt`

- `evidence/sql-injection-search.txt`
- `evidence/source-analysis.md`

15.1 Evidence handling

Live authentication tokens, passwords, and private keys are excluded from this report. Original evidence files should remain unchanged. Encrypted laboratory artifacts were not decrypted.

16 Limitations

- The target is an intentionally vulnerable educational application.
- Testing was limited to the local Docker instance on 24 September 2026.
- No destructive or denial-of-service testing was performed.
- Product-search testing showed SQL errors, not reliable extraction.
- Several source-review items were not dynamically validated.
- Results do not transfer automatically to other Juice Shop versions or to unrelated applications.

17 Conclusion

In the tested local Juice Shop build, authentication could be bypassed through SQL injection, registration accepted an administrative role, and basket access lacked object ownership checks. Public FTP listing and verbose errors added information disclosure. Product search used unsafe SQL construction, with error-level confirmation only.

Remediation should start with parameterized authentication queries, server-controlled roles, and object-level authorization. File exposure and error handling should follow. Source-level observations require separate laboratory validation before they are treated as confirmed.

A Key Laboratory Commands

Commands below are representative of the recorded tests. Secrets are placeholders.

A.1 Nmap

```
1 nmap -sV -p 3000 127.0.0.1 -oN scans/nmap-3000.txt
```

A.2 HTTP baseline

```
1 curl -i http://127.0.0.1:3000
```

A.3 FTP listing

```
1 curl -i http://127.0.0.1:3000/ftp/
```

A.4 Authentication test

```
1 curl -i -X POST http://127.0.0.1:3000/rest/user/login \  
2   -H "Content-Type: application/json" \  
3   --data-binary "@login-sqli-test.json"
```

A.5 Basket authorization test

```
1 curl -i http://127.0.0.1:3000/rest/basket/1 \  
2   -H "Authorization: Bearer <REDACTED_LAB_TOKEN>"
```

A.6 Mass assignment test

```
1 curl -i -X POST http://127.0.0.1:3000/api/Users/ \  
2   -H "Content-Type: application/json" \  
3   --data-binary "@mass-assignment-test.json"
```

B Sensitive Evidence Handling

Do not include in a public portfolio copy:

- Live authentication tokens
- Passwords or password hashes
- Private cryptographic keys
- Decrypted contents of encrypted laboratory artifacts
- Unredacted session dumps

C Report Classification

CONFIDENTIAL

Educational / Portfolio Security Assessment

Prepared by Aryan Singh
Assessment date: 24 September 2026

Local laboratory target only. No production systems tested.